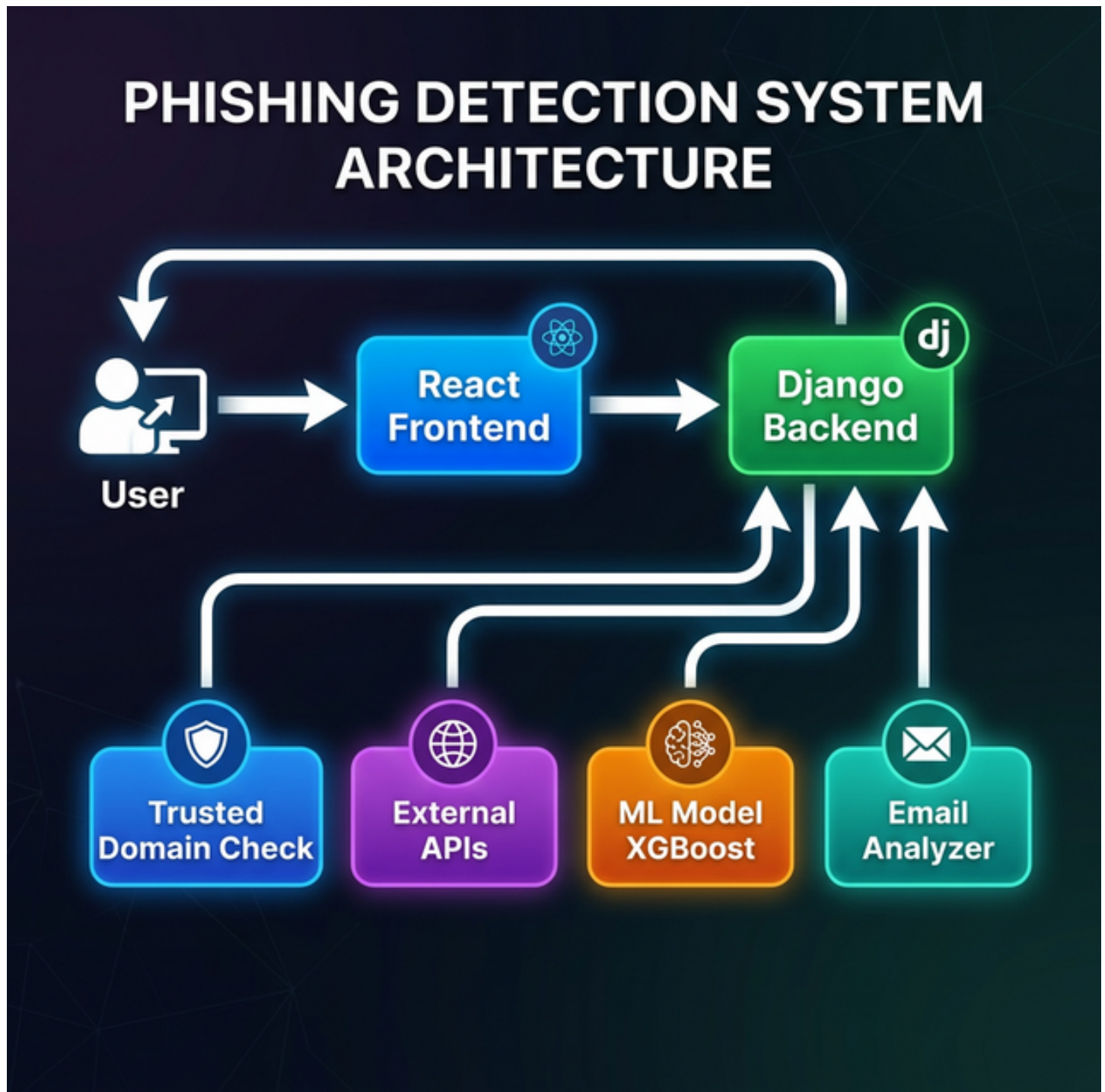


PhishGuard How It Works (In Simple Terms)

PhishGuard is a web application that tells you whether a URL or email is a phishing scam. Think of it like a security guard for links -- you give it a URL, and it runs it through multiple layers of checks to decide: "Is this safe, or is someone trying to trick you?"

The Big Picture



The app has 3 main parts: a React frontend (what you see), a Django backend (the brain), and a machine-learning model (the AI detective).

Part 1: The Frontend (React)

What it is: The web interface the user interacts with -- built with React.

Pages:

Page	Route	What it does
Home	/	Landing page with an intro to PhishGuard
Check URL	/checkurl	Enter a URL -> get phishing/legitimate verdict
Email Analysis	/email-analysis	Paste email headers/body -> get risk assessment
Dashboard	/dashboard	See stats of all scans (totals, charts, recent history)
Features	/features	Info page about the app's capabilities

How it works: When you type a URL and hit "Check", the frontend sends a GET request to the backend at `http://localhost:8000/api/?url=<your-url>`. The backend processes it and sends back a JSON response with the verdict, confidence scores, and which detection method was used.

Part 2: The Backend (Django) The Brain

This is where all the intelligence lives. When a URL comes in, the backend runs it through multiple layers of defense, like a series of security checkpoints.

Layer 1: Emergency & Trusted Domain Whitelist

TIP: *In plain English: "Is this obviously safe?"*

Before doing anything complex, the backend checks if the URL belongs to a major trusted domain like google.com, youtube.com, amazon.com, etc.

- If yes -> immediately return "Legitimate" with 99% confidence
- If no -> move to the next layer

This prevents false positives (i.e., accidentally flagging Google as phishing).

Files: `views.py` (emergency override), `reputation_check.py` (trusted list of ~80 domains)

Layer 2: Reputation Checks (External APIs + Cache)

TIP: In plain English: "Has anyone else already reported this as dangerous?"

The system checks the URL against multiple external threat intelligence sources:

1. Local Cache -- Did we check this URL recently? If so, reuse the result (valid for 24 hours)
2. Typosquatting Detection -- Is the domain suspiciously similar to a trusted brand? e.g., g00gle.com trying to impersonate google.com. Uses Levenshtein distance to measure how "close" two domain names are
3. Homograph Attack Detection -- Is the domain using Unicode tricks? e.g., using a Cyrillic "a" instead of a Latin "a"
4. VirusTotal API -- Sends the URL to VirusTotal where 70+ antivirus engines scan it. If $\geq 5\%$ of engines flag it -> it's phishing
5. Google Safe Browsing API -- Queries Google's real-time database of known malware/phishing sites
6. PhishTank Database -- A background thread downloads a fresh list of known phishing sites every 24 hours

If any of these return a definitive answer, that answer is used. Otherwise, the system falls back to the ML model.

Files: reputationcheck.py, externalapis.py, typosquatting.py, tasks.py

Layer 3: ML Model Prediction (XGBoost)

TIP: In plain English: "Let me analyze the URL's structure to see if it looks fishy."

If no external source gave a definitive answer, the URL is analyzed by an XGBoost machine learning model. The model was trained on a dataset of known phishing and legitimate URLs, and it looks at 15 features extracted from the URL:

#	Feature	What it checks	Phishing sign
1	IP Address	Does the URL use an IP instead of a domain name?	http://192.168.1.1/login
2	Long URL	Is the URL unusually long (>54 chars)?	Long URLs hide malicious paths
3	URL Shortener	Uses bit.ly, tinyurl, etc.?	Hides the real destination
4	@ Symbol	Contains @?	Tricks the browser about the real host
5	// Redirection	Has // after the protocol?	Redirects to a different site
6	Prefix/Suffix -	Has hyphens in the domain?	secure-google-login.com
7	Sub-domains	How many dots in the domain?	login.secure.google.evil.com
8	HTTPS Token	Does "https" appear in domain itself?	https-google.com (not real HTTPS)
9	Age of Domain	How old is the domain registration?	New domains < 6 months are suspicious
10	DNS Record	Does the domain have a valid WHOIS record?	No record = suspicious
11	Web Traffic	Does the site have traffic (Alexa rank)?	No traffic = suspicious
12	Registration Length	How long until domain expires?	Short registration ≤ 1 year
13	Statistical Report	Is the domain/IP in known phishing lists?	Known bad actors

#	Feature	What it checks	Phishing sign
14	iFrame	Does the page use hidden iFrames?	Embeds malicious content
15	Mouse Over	Does the page have onmouseover scripts?	Hides real links on hover

These 15 numbers are fed into the XGBoost model, which outputs a probability (e.g., 87% phishing, 13% legitimate).

Files: feature.py (feature extraction), trainnewmodel.py (model training), apps.py (model loading)

Layer 4: Email Analysis (Separate Feature)

TIP: In plain English: "Does this email look like a scam?"

The app also lets users paste email content to analyze it for phishing. This is a completely separate analysis from URL checking:

Header Analysis (30% weight):

- SPF/DKIM/DMARC authentication checks
- Reply-To mismatch detection (From ? Reply-To)
- Sender domain lookalike/typosquatting detection
- Unusual email routing patterns

Content Analysis (70% weight):

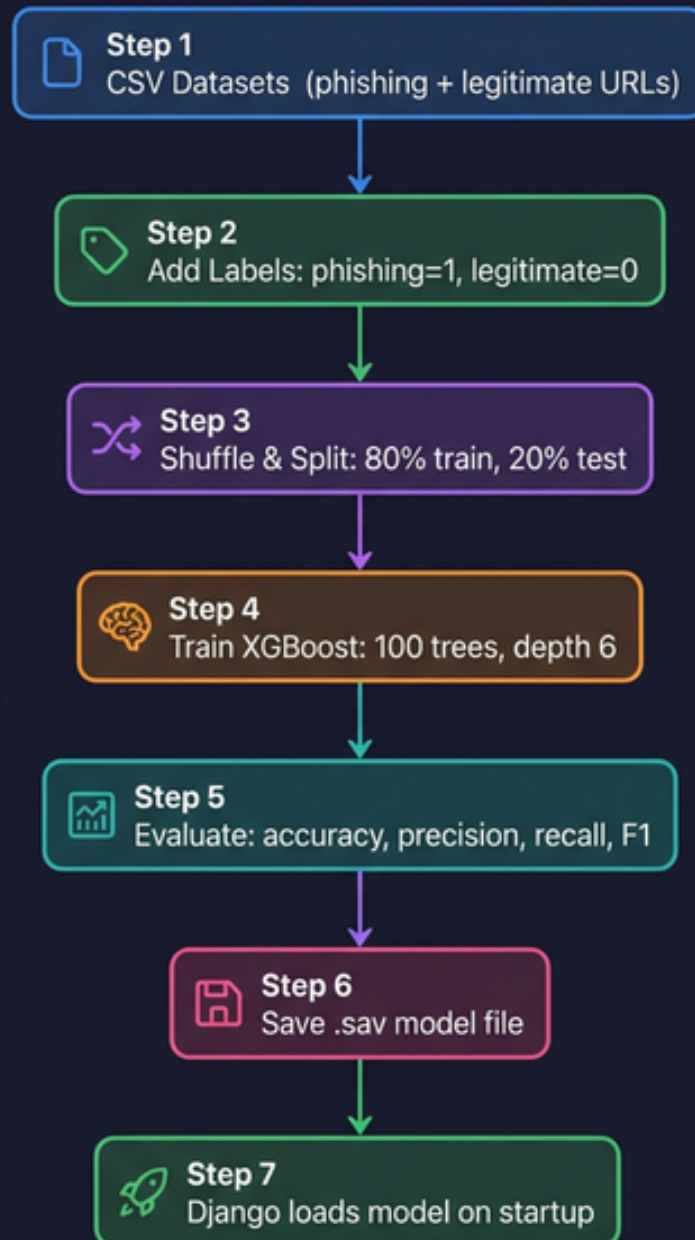
- Urgency keywords ("act now", "suspend", "verify immediately")
- Suspicious sender patterns (random strings, free email providers)
- URL extraction and analysis from the body
- Suspicious TLD detection (.tk, .xyz, .buzz, etc.)

The two scores are combined into a risk score (0-100) and a risk level: Safe, Low Risk, Medium Risk, or High Risk.

File: email_analysis.py (860 lines -- the most complex module)

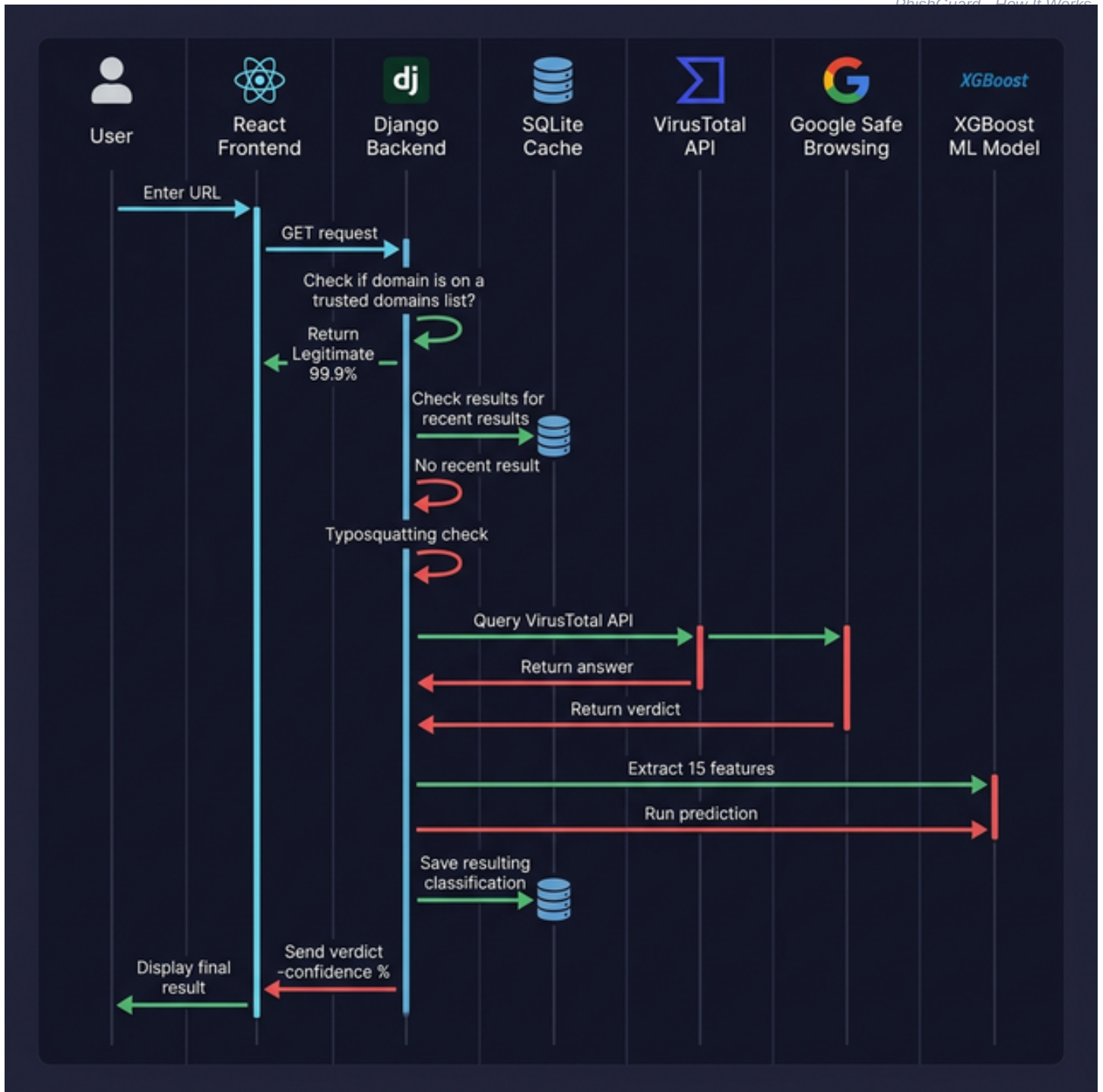
Part 3: The ML Pipeline

The ML training is done offline in trainnewmodel.py:



The model is saved as a .sav pickle file and Django loads it into memory when the server starts. If the model file is missing, a fallback RandomForest model is created from synthetic patterns (hardcoded in apps.py).

Data Flow: What Happens When You Check a URL



Database & Storage

What	Where	Purpose
Scan history	db.sqlite3 (Django ORM)	Stores every scan result for the dashboard
Reputation cache	cache/reputation_cache.db (SQLite)	Caches URL/domain reputation for 24h
ML model	XGBoostClassifier.sav (pickle)	The trained model loaded into memory

What	Where	Purpose
Phishing DB	Updated via background thread	Fresh phishing list from PhishTank (every 24h)

Summary

PhishGuard uses a defense-in-depth approach with 4 layers:

1. Whitelist -> Instantly clear known-good domains
2. External Intelligence -> Ask VirusTotal, Google Safe Browsing, and PhishTank
3. ML Model -> Analyze 15 URL characteristics with XGBoost
4. Email Analysis -> Scan headers and content for phishing patterns

Each layer adds confidence. The result is returned to the user with a clear verdict (phishing/legitimate) and a confidence percentage, so users know how certain the system is.